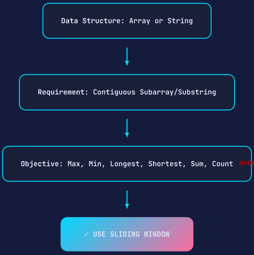
Sliding Window

Master the Pattern with Interactive Visualizations

🎯 Pattern Identification

Before applying the Sliding Window technique, verify these three conditions:



Data Structure: Array or String

↓

Requirement: Contiguous Subarray/Substring

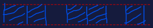
↓

Objective: Max, Min, Longest, Shortest, Sum, Count, average, at most k, at least k, exactly k

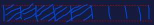
↓

✓ USE SLIDING WINDOW

subsequence can be continuous/
or non-continuous blocks



Subarrays/Substrings are continuous
blocks



Key Keywords to Watch For:

- "At most K" / "At least K" / "Exactly K"
- "Maximum sum" / "Minimum length"
- "Longest substring" / "Shortest subarray"
- "Contiguous elements"

sum, count, average

4 Steps to solve
1. Identify the pattern
2. Determine if it is a fixed or a variable window
3. Find its data/information
   example: sum, count, average, etc
4. Find out the information for the new window

📊 Two Types of Windows

Type A: Fixed Size Window

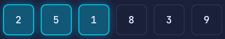
applied when window size is given for the subarray/substring

Situation: Window size is specified (e.g., "subarray of size K")

Example: Find max sum of subarray with size K=3

Array: [2, 5, 1, 8, 3, 9]

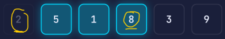
STAGE 0: CALCULATE FIRST WINDOW



Window [0-2]: 2 + 5 + 1 = 8
maxSum = 8

STAGE 1: SLIDE WINDOW (REMOVE 2, ADD 8)

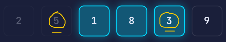
Window will expand from the right and contract from the left



Window [1-3]: sum = 8 - 2 + 8 = 14
maxSum = 14 ✓ (better!)

STAGE 2: SLIDE WINDOW (REMOVE 5, ADD 3)

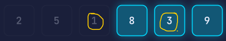
Window will expand from the right and contract from the left



Window [2-4]: sum = 14 - 5 + 3 = 12
maxSum = 14 (current window not better)

STAGE 3: SLIDE WINDOW (REMOVE 1, ADD 9)

Window will expand from the right and contract from the left



Window [3-5]: sum = 12 - 1 + 9 = 20
maxSum = 20 ✓ (new maximum!)

> 🎯 **Result:** Maximum sum = 20 (window [8, 3, 9])
> **Key Insight:** We slide in O(1) time by removing the element leaving and adding the element entering, instead of recalculating the entire sum each time!

JAVA

```java
/**
 * FIXED WINDOW PATTERN
 * Problem: Find maximum sum of subarray with size K
 *
 * Visual Example: arr = [2, 3, 1, 4, 5], k = 3
 * Step 1: [2, 3, 1] → sum = 6
 * Step 2: [3, 1, 4] → sum = 8 (remove 2, add 4)
 * Step 3: [1, 4, 5] → sum = 10 (remove 3, add 5)
 *
 * Time Complexity: O(n)
 * Space Complexity: O(1)
 */
public int maxSumSubarray(int[] arr, int k) {
    // Edge case: array smaller than window size
    if (arr.length < k) return -1;

    // STEP 1: Calculate first window sum [0 to k-1]
    int sum = 0;
    for (int i = 0; i < k; i++) {
        sum += arr[i];
    }

    // Initialize max with first window
    int maxSum = sum;
    int low = 0;        // Left boundary of window
    int high = k;       // Points to next element to include

    // STEP 2: Slide the window one position at a time
    while (high < arr.length) {
        // 🔑 KEY INSIGHT: Subtract element leaving, add element entering
        // This gives O(1) per slide instead of O(k)
        sum = sum - arr[low] + arr[high];

        // Update maximum if current window is better
        maxSum = Math.max(maxSum, sum);

        // Move both pointers forward (maintain window size k)
        low++;
        high++;
    }

    return maxSum;
}
```

Type B: Variable Size Window

Situation: Window size is NOT specified, it expands and shrinks dynamically

> 🔄 **The Hiring & Firing Analogy:**
>
> **Expand (high++):** "Hire" new elements to achieve your goal
> **Shrink (low++):** "Fire" elements to optimize or maintain validity

📈 Finding LONGEST / MAXIMUM Window

Use Case: "Longest substring with K distinct characters" or "Maximum subarray with sum ≤ K"

> **Core Logic:** Shrink ONLY when window becomes INVALID
>
> 1. Expand high pointer (add element)
> 2. Check if window is INVALID (e.g., > K distinct chars)
> 3. While INVALID: Shrink low pointer until valid again
> 4. Update maximum result

Stage-by-Stage: Longest Substring with K=2 Distinct Characters

String: "AABAC" | Goal: Find longest substring with at most 2 distinct characters

STAGE 0: START WITH FIRST CHARACTER

A A B A C

Window: [A] | low=0, high=0
Map: {A:1} | Distinct: 1 ≤ 2 ✓ VALID
maxLen = 1

STAGE 1: EXPAND - ADD 'A'

A A B A C

Window: [A, A] | low=0, high=1
Map: {A:2} | Distinct: 1 ≤ 2 ✓ VALID
maxLen = 2

STAGE 2: EXPAND - ADD 'B'

A A B A C

Window: [A, A, B] | low=0, high=2
Map: {A:2, B:1} | Distinct: 2 ≤ 2 ✓ VALID
maxLen = 3

STAGE 3: EXPAND - ADD 'A'

A A B A C

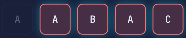
Window: [A, A, B, A] | low=0, high=3
Map: {A:3, B:1} | Distinct: 2 ≤ 2 ✓ VALID
maxLen = 4

STAGE 4: EXPAND - ADD 'C' (INVALID!)

A A B A C

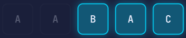
Window: [A, A, B, A, C] | low=0, high=4
Map: {A:3, B:1, C:1} | Distinct: 3 > 2 ✗ INVALID
Must shrink!

STAGE 5: SHRINK - REMOVE 'A' (STILL INVALID)



Window: [A, B, A, C] | low=1, high=4
Map: {A:2, B:1, C:1} | Distinct: 3 > 2 ✗ STILL INVALID
Keep shrinking!

STAGE 6: SHRINK - REMOVE 'A' (NOW VALID!)



Window: [B, A, C] | low=2, high=4
Map: {B:1, A:1, C:1} → Remove C, keep {B:1, A:1}
Distinct: 2 ≤ 2 ✓ VALID AGAIN
maxLen = 4 (no update, current window = 3)

> 🎯 **Result:** Maximum length = 4 (window [A, A, B, A])
> **Key Pattern:** We only shrink when the window becomes INVALID (too many distinct characters). Once valid again, we continue expanding!

JAVA

```java
/**
 * VARIABLE WINDOW - MAXIMUM PATTERN
 * Problem: Longest substring with K distinct characters
 *
 * Visual Example: s = "AABACBEBE", k = 2
 *
 * Step-by-Step:
 * [A] → {A:1} → size=1, valid (1≤2) → maxLen=1
 * [A,A] → {A:2} → size=1, valid → maxLen=2
 * [A,A,B] → {A:2,B:1} → size=2, valid (2≤2) → maxLen=3
 * [A,A,B,A] → {A:3,B:1} → size=2, valid → maxLen=4
 * [A,A,B,A,C] → {A:3,B:1,C:1} → size=3, INVALID! (3>2)
 *   Shrink: Remove A → {A:2,B:1,C:1} → still invalid
 *   Shrink: Remove A → {B:1,C:1} → valid! Window=[B,A,C]
 *
 * 🔑 KEY PATTERN: Shrink ONLY when window becomes INVALID
 *
 * Time: O(n), Space: O(k)
 */
public int longestSubstringKDistinct(String s, int k) {
    // Edge case: k=0 means no characters allowed
    if (k == 0) return 0;

    // Map tracks frequency of characters in current window
    Map<Character, Integer> map = new HashMap<>();
    int low = 0, maxLen = 0;

    // high pointer expands the window
    for (int high = 0; high < s.length(); high++) {
        char c = s.charAt(high);

        // EXPAND: Add new character to window
        map.put(c, map.getOrDefault(c, 0) + 1);

        // SHRINK: Only when INVALID (more than K distinct chars)
        // ⚠️ CRITICAL: This is "while", not "if" - keep shrinking until valid
        while (map.size() > k) {
            char leftChar = s.charAt(low);

            // Decrease frequency of leftmost character
            map.put(leftChar, map.get(leftChar) - 1);

            // Remove from map if count reaches 0 (important for size check)
            if (map.get(leftChar) == 0) {
                map.remove(leftChar);
            }

            // Move left boundary forward
            low++;
        }

        // UPDATE RESULT: Window is guaranteed valid here
        // Calculate current window length
        maxLen = Math.max(maxLen, high - low + 1);
    }

    return maxLen;
}
```

Step-by-Step Visualization

1 **String: [A, A, B, A, C], K=2**
Add A → Map: {A:1} → Valid (1 ≤ 2) → maxLen = 1

2 **Window: [A, A]**
Add A → Map: {A:2} → Valid (1 ≤ 2) → maxLen = 2

3 **Window: [A, A, B]**
Add B → Map: {A:2, B:1} → Valid (2 ≤ 2) → maxLen = 3

4 **Window: [A, A, B, A]**
Add A → Map: {A:3, B:1} → Valid (2 ≤ 2) → maxLen = 4

5 **Window: [A, A, B, A, C]**
Add C → Map: {A:3, B:1, C:1} → INVALID (3 > 2)
Shrink: Remove A → Map: {A:2, B:1, C:1} → Still invalid
Shrink: Remove A → Map: {B:1, C:1} → Valid → Window: [B, A, C], length = 3

📉 Finding SHORTEST / MINIMUM Window

Use Case: "Minimum subarray with sum ≥ Target" or "Smallest window containing all characters"

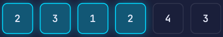
Core Logic: Shrink WHILE window is VALID (opposite of maximum!)

1. Expand high pointer (add element)
2. Check if window is VALID (e.g., sum ≥ target)
3. While VALID: Record result, then shrink low to find smaller window
4. Keep shrinking until window becomes invalid

Stage-by-Stage: Minimum Subarray Sum ≥ 7

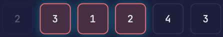
Array: [2, 3, 1, 2, 4, 3] | Goal: Find shortest subarray with sum ≥ 7

STAGES 0-2: EXPAND UNTIL SUM ≥ 7



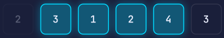
Window: [2, 3, 1, 2] | low=0, high=3
Sum: 2 + 3 + 1 + 2 = 8 ≥ 7 ✓ VALID!
minLen = 4 (first valid window found!)

STAGE 3: SHRINK - REMOVE 2 (BECOMES INVALID)



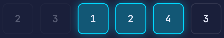
Window: [3, 1, 2] | low=1, high=3
Sum: 8 - 2 = 6 < 7 ✗ INVALID
Lost validity! Stop shrinking, continue expanding.

STAGE 4: EXPAND - ADD 4



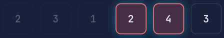
```
Window: [3, 1, 2, 4] | low=1, high=4
Sum: 6 + 4 = 10 ≥ 7 ✓ VALID!
minLen = 4 (still 4, no improvement)
```

STAGE 5: SHRINK - REMOVE 3 (STILL VALID!)



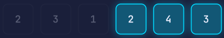
```
Window: [1, 2, 4] | low=2, high=4
Sum: 10 - 3 = 7 ≥ 7 ✓ STILL VALID!
minLen = 3 ✓ (found smaller valid window!)
```

STAGE 6: SHRINK - REMOVE 1 (BECOMES INVALID)



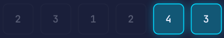
```
Window: [2, 4] | low=3, high=4
Sum: 7 - 1 = 6 < 7 ✗ INVALID
Lost validity! Continue expanding.
```

STAGE 7: EXPAND - ADD 3



```
Window: [2, 4, 3] | low=3, high=5
Sum: 6 + 3 = 9 ≥ 7 ✓ VALID!
minLen = 3 (no change)
```

STAGE 8: SHRINK - REMOVE 2 (STILL VALID!)



```
Window: [4, 3] | low=4, high=5
Sum: 9 - 2 = 7 ≥ 7 ✓ STILL VALID!
minLen = 2 ✓ BEST ANSWER!
```

> 🎯 **Result:** Minimum length = 2 (window [4, 3])
> **Key Pattern:** We shrink WHILE the window is VALID to find the smallest valid window. Only when it becomes invalid do we stop and expand again!

JAVA

```java
/**
 * VARIABLE WINDOW - MINIMUM PATTERN
 * Problem: Find minimum length subarray with sum ≥ target
 *
 * Visual Example: nums = [2,3,1,2,4,3], target = 7
 *
 * The "Hiring & Firing" Process:
 * Add 2: [2] sum=2 < 7 (not enough, keep hiring)
 * Add 3: [2,3] sum=5 < 7 (not enough)
 * Add 1: [2,3,1] sum=6 < 7 (not enough)
 * Add 2: [2,3,1,2] sum=8 ≥ 7 ✓ VALID! minLen=4
 *   Can we do better? Try firing...
 *   Fire 2: [3,1,2] sum=6 < 7 ✗ (lost validity, stop firing)
 * Add 4: [3,1,2,4] sum=10 ≥ 7 ✓ VALID! minLen=4
 *   Fire 3: [1,2,4] sum=7 ≥ 7 ✓ Still valid! minLen=3
 *   Fire 1: [2,4] sum=6 < 7 ✗ (stop firing)
```

```
 * Add 3: [2,4,3] sum=9 ≥ 7 ✓ VALID!
 *   Fire 2: [4,3] sum=7 ≥ 7 ✓ Still valid! minLen=2 🎯
 *
 * 🔑 KEY PATTERN: Shrink WHILE window is VALID (opposite of maximum!)
 *
 * Time: O(n), Space: O(1)
 */
public int minSubArrayLen(int target, int[] nums) {
    int low = 0, high = 0, sum = 0;
    // Start with "infinity" - if we never find valid window, return 0
    int minLen = Integer.MAX_VALUE;

    // high pointer expands the window
    while (high < nums.length) {
        // EXPAND: "Hire" new element (add to sum)
        sum += nums[high];

        // SHRINK: WHILE window is VALID (sum meets target)
        // ⚠️ CRITICAL DIFFERENCE: Shrink WHILE valid, not while invalid!
        // We keep "firing" as long as work still gets done
        while (sum >= target) {
            // Calculate current window length
            int currentLen = high - low + 1;

            // UPDATE RESULT: Record this valid window if it's smaller
            minLen = Math.min(minLen, currentLen);

            // Try to shrink further - "Fire" leftmost element
            sum -= nums[low];
            low++;
            // Loop continues if still valid, stops when sum < target
        }

        // Move to next element
        high++;
    }

    // If minLen never updated, no valid window exists
    return (minLen == Integer.MAX_VALUE) ? 0 : minLen;
}
```

⚖️ Pattern Comparison

📈 MAXIMUM Pattern Logic

```
✓ Window is VALID
  → Keep expanding
  → Update max result

✗ Window becomes INVALID
  → Shrink until valid
  → Then continue expanding
```

Goal: Make window as LARGE as possible while staying valid

📉 MINIMUM Pattern Logic

```
✗ Window is INVALID
  → Keep expanding
  → Looking for valid window

✓ Window becomes VALID
  → Update min result
  → Shrink to find smaller
```

Goal: Make window as SMALL as possible while staying valid

Aspect	Fixed Window	Variable - Maximum	Variable - Minimum
Window Size	Constant (K)	Dynamic	Dynamic
Expand Logic	Slide by 1	high++	high++
Shrink Trigger	Slide by 1	When INVALID	While VALID
Shrink Goal	Maintain size K	Make valid again	Find smaller valid
Result Update	Every slide	When valid	While valid
Loop Condition	N/A	while (condition violated)	while (condition satisfied)
Example	Max sum size K	Longest K distinct	Min sum ≥ target

🚀 Advanced: Minimum Window Substring

Problem: Find the smallest substring in S that contains all characters of T (including duplicates)

> **Pattern Type:** Variable Window - Minimum (shrink while valid)
>
> **Key Insight:** Use frequency arrays to track what we NEED (from T) and what we HAVE (current window in S)

Algorithm Steps

1 **Setup:** Create two frequency arrays of size 256 (ASCII)
- need[]: Count of each character in string T
- have[]: Count of each character in current window of S

2 **Expand:** Move high pointer, add character to have[] array

3 **Check Validity:** Use helper function to verify if have[i] ≥ need[i] for all characters

4 **Shrink While Valid:** Record minimum window, then remove character at low pointer to try for smaller window

5 **Return Result:** Substring from stored start index with minimum length

JAVA

```java
/**
 * ADVANCED: MINIMUM WINDOW SUBSTRING (Amazon Interview Question)
 * Problem: Find smallest substring in S containing all characters of T
 *
 * Visual Example: S = "ADOBECODEBANC", T = "ABC"
 *
 * Setup:
 * need[] = {A:1, B:1, C:1} - what we need from T
 * have[] = {tracking current window} - what we currently have
 *
 * Process:
 * Window "ADOBEC":
 *   have = {A:1,D:1,O:1,B:1,E:1,C:1} → VALID (has all A,B,C)
 *   Length = 6, record this!
 *   Try shrinking: Remove A → "DOBEC" → INVALID (missing A)
 *
 * Continue expanding...
 * Window "ODEBANC":
 *   have = {O:1,D:1,E:1,B:1,A:1,N:1,C:1} → VALID
 *   Length = 7, not better than 6
 *
 * Final answer: "BANC" (length 4) 🎯
 *
 * 🔑 KEY INSIGHT: This is MINIMUM pattern - shrink WHILE valid
 *
 * Time: O(|S| + |T|), Space: O(1) - fixed size arrays
 */
public String minWindow(String s, String t) {
    // Edge cases
    if (s.length() == 0 || t.length() == 0) return "";

    // Use arrays of size 256 for all ASCII characters
    // More efficient than HashMap for character counting
    int[] need = new int[256]; // What we need (from T)
    int[] have = new int[256]; // What we have (current window in S)

    // STEP 1: Populate 'need' array from string T
    // Example: T="ABC" → need['A']=1, need['B']=1, need['C']=1
    for (char c : t.toCharArray()) {
        need[c]++;
    }

    // Track minimum window position and length
    int low = 0, minLen = Integer.MAX_VALUE, startInd = -1;

    // STEP 2: Expand window with high pointer
    for (int high = 0; high < s.length(); high++) {
        // EXPAND: Add character to current window
        have[s.charAt(high)]++;

        // STEP 3: SHRINK WHILE window is VALID
        // This is the MINIMUM pattern - keep shrinking while we can
        while (isValid(have, need)) {
            // Calculate current window length
            int currentLen = high - low + 1;

            // UPDATE RESULT: If this window is smaller, record it
            if (currentLen < minLen) {
                minLen = currentLen;
                startInd = low;  // Remember where this window starts
            }
```

```
            // Try to shrink: Remove leftmost character
            have[s.charAt(low)]--;
            low++;
            // Loop continues if still valid, stops when invalid
        }
    }

    // STEP 4: Return result
    // If minLen never updated, no valid window exists
    return minLen == Integer.MAX_VALUE ? "" : s.substring(startInd, startInd + minLen);
}

/**
 * Helper function: Check if current window contains all needed characters
 * Valid when: have[i] >= need[i] for ALL characters
 *
 * Example:
 * need = {A:1, B:1, C:1}, have = {A:2, B:1, C:1, D:1}
 * Result: TRUE (we have enough of each needed character)
 *
 * need = {A:1, B:1, C:1}, have = {A:1, B:0, C:1}
 * Result: FALSE (missing B)
 */
private boolean isValid(int[] have, int[] need) {
    // Check all 256 ASCII characters
    for (int i = 0; i < 256; i++) {
        // If we need this character but don't have enough
        if (have[i] < need[i]) {
            return false;
        }
    }
    // We have enough of everything!
    return true;
}
```

🎓 Common Problem Patterns

Fixed Window

- Max/Min sum of size K
- Average of subarrays
- Find anagrams of length K

Variable - Maximum

- Longest K distinct chars
- Longest without repeating
- Max consecutive ones
- Fruits into baskets

Variable - Minimum

- Min subarray sum ≥ target
- Min window substring
- Smallest covering substring

📌 Quick Reference Guide & Code Templates

When to Use Each Pattern:

Fixed Window: Problem explicitly states window size

Variable Maximum: Keywords like "longest", "maximum", constraints like "at most K" → Shrink when INVALID

Variable Minimum: Keywords like "shortest", "minimum", constraints like "at least K" or "≥ target" → Shrink while VALID

📝 Code Templates

JAVA

```
Template 1: Fixed Window

// Use when: Window size K is given
public int fixedWindowTemplate(int[] arr, int k) {
    int windowSum = 0;
    int result = 0; // or Integer.MIN_VALUE/MAX_VALUE

    // Calculate first window
    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }
    result = windowSum;

    // Slide window
    for (int i = k; i < arr.length; i++) {
        windowSum = windowSum - arr[i - k] + arr[i];
        result = Math.max(result, windowSum); // or min, or other operation
    }

    return result;
}
```

JAVA

```java
Template 2: Variable Window - MAXIMUM/LONGEST

// Use when: Finding longest/maximum with constraint "at most K" or "≤ K"
public int variableMaxTemplate(int[] arr, int k) {
    Map<Integer, Integer> map = new HashMap<>(); // or other data structure
    int low = 0, maxResult = 0;

    for (int high = 0; high < arr.length; high++) {
        // 1. EXPAND: Add arr[high] to window
        // Update map/sum/count...

        // 2. SHRINK: While window is INVALID (condition violated)
        while (/* window is invalid */) {
            // Remove arr[low] from window
            // Update map/sum/count...
            low++;
        }

        // 3. UPDATE RESULT: Window is guaranteed valid here
        maxResult = Math.max(maxResult, high - low + 1);
    }

    return maxResult;
}

// 🎯 REMEMBER: Shrink when INVALID to restore validity
// Examples:
// - while (map.size() > k)           // Too many distinct
// - while (sum > target)              // Sum exceeds limit
// - while (windowLength - maxFreq > k) // Too many replacements needed
```

JAVA

```java
Template 3: Variable Window - MINIMUM/SHORTEST

// Use when: Finding shortest/minimum with constraint "at least K" or "≥ target"
public int variableMinTemplate(int[] arr, int target) {
    int low = 0, sum = 0;
    int minResult = Integer.MAX_VALUE;

    for (int high = 0; high < arr.length; high++) {
        // 1. EXPAND: Add arr[high] to window
        sum += arr[high];

        // 2. SHRINK: While window is VALID (condition satisfied)
        while (/* window is valid */) {
            // 3. UPDATE RESULT: Record this valid window
            minResult = Math.min(minResult, high - low + 1);

            // Try to shrink further
            sum -= arr[low];
            low++;
        }
    }

    return minResult == Integer.MAX_VALUE ? 0 : minResult;
}

// 🎯 REMEMBER: Shrink WHILE VALID to find smallest valid window
// Examples:
// - while (sum >= target)           // Sum meets requirement
// - while (isValid(have, need))     // Contains all required chars
// - while (count == k)              // Has exactly k elements
```

🧠 Memory Aid: The Golden Rule

For MAXIMUM/LONGEST problems:

Think: "I want the window as BIG as possible"

→ Only shrink when forced to (when INVALID)

For MINIMUM/SHORTEST problems:

Think: "I want the window as SMALL as possible"

→ Keep shrinking while possible (while VALID)